```
        self.fields["musician"].help_text = \
            "Fill in if you are a musician seeking a band"
        self.fields["band"].help_text = \
            "Fill in if you are a band seeking a musician"
```

Modify the help text for the musician field.

Using a `ModelForm` doesn't only mean you have less code to write—it also adds some verification in the background. When initially writing the code for `SeekingAdForm`, I accidentally wrote `seeing` instead of `seeking` in the `fields` list. When I ran the Django server, an error got issued, as the named field did not match an attribute of the `SeekingAd` model.

It is generally recommended to explicitly list the form's fields. There is a special value, `fields = "__all__"`, that can be used instead of a list to save you some typing, but I recommend against using this. If you change your model in the future, you might accidentally add a field into the form that should have got excluded. There have been cases where companies have exposed information they shouldn't have to the public through exactly this kind of bug.

An alternative to using the `fields` attribute is `except`. This is also a list, but here you list the fields that don't participate in the form. This has the same danger as `__all__`, where additions to the model automatically show up in the form, so again, this is probably not the best choice.

A shorter shortcut

If the only thing you need to do with your `ModelForm` is associate it with a `Model` class and specify the `fields` attribute, you can get there with even less code. The `modelform_factory` function is a factory that creates `ModelForm` classes. The typical use takes two arguments: a reference to the `Model` and a list of fields:

```
from django import forms

SeekingAdForm = forms.modelform_factory(SeekingAd, fields = ["seeking",
    "musician", "band", "content", ]
```

The factory takes other arguments as well. Like with the class, you can use `except` instead of `fields`; specify the labels, widgets, help text, and error messages; and more. For more information, see https://docs.djangoproject.com/en/dev/ref/forms/models/.

Instead of overriding the `.__init__()` method, there are several dictionaries you can use inside the `Meta` inner class. The `labels`, `help_texts`, `widgets`, and `error_messages` attributes all override the corresponding item for a field. For example, the following is an alternate way of overriding the label for the `seeking` field:

```
class SeekingAdForm(forms.ModelForm):
    class Meta:
        model = SeekingAd
        fields = ["seeking", "musician", "band", "content", ]
```